

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3

6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3

14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3

22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Solutions:

1. Testing as an Engineering Activity

Testing should be treated as a systematic engineering activity rather than random execution of test cases. The startup should first analyze requirements, identify risks, define test objectives, and design test cases systematically. Techniques such as equivalence partitioning, boundary value analysis, decision table testing, and state transition testing can be used to cover different input conditions. For example, if a banking application accepts an amount between ₹100 and ₹50,000, testing should include values below, at, and above these limits. Test cases should contain inputs, expected results, actual results, and execution status, while defects should be recorded systematically. This structured approach provides better test coverage, reduces duplicate testing, identifies defects earlier, and produces repeatable results. Therefore, engineering-based testing is more effective than random testing because every test is designed with a specific quality objective.

2. Defined Testing Process in a Banking System

A well-defined testing process improves software quality by ensuring that testing activities are planned, executed, documented, and reviewed consistently. In a banking system, the process should include requirement analysis, test planning, test case design, unit testing, integration testing, system testing, regression testing, and user acceptance testing. For example, during fund transfer testing, valid transfers, insufficient balance, invalid account numbers, transaction limits, network failures, and duplicate transactions should all be tested. Defects should be recorded with clear severity, priority, steps to reproduce, expected result, and actual result. A defined process reduces missed scenarios, improves communication between developers and testers, and ensures that important banking functions are repeatedly validated after changes. Thus, process standardization increases reliability, reduces defect leakage, and improves customer confidence in the banking system.

3. Testing as a Structured SDLC Process

Testing should be performed as a continuous and structured process throughout the SDLC rather than only after coding. During requirement analysis, testers identify unclear and missing requirements; during design, they prepare test scenarios; during development, unit testing verifies individual components; during integration testing, interactions between modules are tested; and during system testing, the complete application is validated. For an e-commerce platform, login, product search, cart, payment, and order modules should first be tested individually and then together. Regression testing should be performed whenever changes are introduced. Process discipline ensures that requirements are traceable to test cases and

defects are systematically reported and resolved. Therefore, structured testing across SDLC phases detects defects earlier, reduces integration failures, improves coverage, and produces better-quality software.

4. Error, Defect, and Failure

An error is a human mistake made during software development, a defect is the fault introduced into the software because of that mistake, and a failure occurs when the defect is executed and produces an incorrect result. For example, if a developer misunderstands the login requirement and writes incorrect password-validation logic, the misunderstanding is an error and the incorrect code is a defect. When a user enters an invalid password and the system still allows login, the observed incorrect behavior is a failure. These terms are related because an error can introduce a defect, and the defect can cause a failure during execution. Correctly identifying these concepts helps testers report issues clearly and helps developers locate the root cause. Therefore, understanding error, defect, and failure improves defect reporting and software quality.

5. Early Testing Principle

The early testing principle states that testing should begin as early as possible in the software development lifecycle because defects become more expensive to fix when discovered at later stages. A tester can review requirements and design documents before coding begins to identify ambiguity, missing conditions, and inconsistencies. For example, if a banking requirement does not specify the maximum fund-transfer amount, a tester can identify this during requirement review instead of allowing developers to implement an incomplete solution. Early detection prevents defects from moving into design and implementation stages, reducing rework and maintenance costs. Therefore, involving testers early improves defect detection efficiency, reduces development effort, and increases overall software quality.

6. Tester Involvement During Requirement Analysis

A tester should participate in requirement analysis to identify ambiguities, missing conditions, contradictions, and requirements that cannot be tested clearly. In a healthcare application, the tester can review requirements related to patient registration, medical records, appointments, and data security and ask questions about undefined conditions. For example, if a requirement states that patient information must be updated but does not specify who can update it or how conflicting information should be handled, the tester can identify the ambiguity before development begins. Clear acceptance criteria and measurable requirements can then be prepared. Early tester involvement prevents misunderstandings from becoming design and coding defects. Therefore, involving testers during requirement analysis supports early defect prevention, reduces rework, and improves the quality and testability of the final healthcare system.

7. Defects Caused by Ambiguous Requirements

Defects can originate at any stage of the software lifecycle, and unclear requirements are a major source of implementation defects. In a payroll system, if the requirement does not clearly specify how overtime, tax, allowances, or deductions should be calculated, developers may interpret the requirement differently. This can result in incorrect salary calculations even if the code works according to the developer's interpretation. Requirement reviews, stakeholder discussions, prototypes, acceptance criteria, and requirement traceability can reduce such problems. Testers should identify ambiguous statements and convert requirements into measurable conditions before development begins. Therefore, clear and complete requirements prevent misunderstandings, reduce functional defects, minimize rework, and ensure that the developed payroll system matches business expectations.

8. Boundary Value Analysis

A crash caused by boundary input values is a functional or validation defect that occurs because the software does not correctly handle inputs at or around specified limits. Such a defect may have been missed because testing focused only on normal values. Boundary Value Analysis (BVA) specifically tests values at the minimum and maximum boundaries and values immediately below and above them. For example, if a system accepts ages from 18 to 60, the tester should test 17, 18, 19, 59, 60, and 61. If the application crashes at 60 or 61, the defect can be detected before deployment. Therefore, boundary value analysis improves test coverage around high-risk input limits and is an effective technique for preventing boundary-related failures.

9. Defect Repository and Test Case Design

A defect repository stores historical information about defects, including their module, type, severity, root cause, release, and resolution. Instead of using the repository only for tracking, the company should analyze historical defect patterns and convert recurring problems into new test cases. For example, if previous releases repeatedly contained login failures due to incorrect password handling, future regression suites should include invalid passwords, empty passwords, locked accounts, and repeated login attempts. Modules with high defect frequency should receive additional testing effort. Root-cause information can also help testers design preventive test scenarios. Therefore, a defect repository acts as a source of testing knowledge, helping improve test coverage, prevent recurring defects, and increase software quality across releases.

10. Functional Defect in ATM Withdrawal

An ATM withdrawal failure caused by incorrect transaction logic is a functional defect because the system does not perform a required business function correctly. For example, if the ATM deducts money from the customer's account without dispensing cash, or dispenses cash without correctly updating the account

balance, the withdrawal functionality is incorrect. This could have been prevented through positive and negative functional testing, including valid withdrawals, insufficient balance, invalid amounts, maximum withdrawal limits, network interruption, and transaction cancellation. State transition testing can also verify states such as transaction initiated, processing, successful, failed, and cancelled. Therefore, systematic functional and transaction testing would have identified the incorrect logic before deployment and improved ATM reliability.

11. Defect Clustering

Defect clustering is the principle that a small number of modules often contain a large proportion of defects. The testing team should analyze historical defect data and identify modules with high defect frequency, complexity, or business risk. For example, in a large application, if payment, authentication, and transaction modules contain most previous defects, these modules should receive higher testing priority than low-risk modules. Additional regression, negative, integration, and exploratory tests can be designed for these areas. This allows limited testing time and resources to be concentrated where defects are most likely to occur. Therefore, defect clustering improves testing efficiency, increases the probability of detecting important defects, and supports risk-based test planning.

12. Performance Defect

Slow performance under heavy user load is classified primarily as a performance defect because the system fails to meet expected response-time, throughput, or scalability requirements. For example, if an online application responds within two seconds for 100 users but takes 20 seconds when 5,000 users access it simultaneously, the system has a performance problem. Load testing can evaluate behavior under expected traffic, while stress testing determines how the system behaves beyond normal capacity. Performance monitoring can identify bottlenecks in the application, database, network, or server resources. Correct classification helps the team select appropriate testing and debugging techniques rather than treating the issue as a normal functional defect. Therefore, defect classification supports faster root-cause identification and more effective resolution.

13. Historical Defect Data for Regression Testing

Historical defect data is highly valuable for regression testing because previous defects indicate areas that are likely to fail again after software changes. Testers should analyze the defect repository and identify recurring defects, high-risk modules, affected features, and previous root causes. These defects should then be converted into permanent regression test cases. For example, if previous releases contained repeated payment failures after updates to the checkout module, payment validation, transaction confirmation,

timeout, and duplicate-payment scenarios should be included in every regression cycle. Regression suites can be prioritized based on defect severity and business risk, and automation can be used for frequently executed cases. Therefore, reusing historical defect data improves test coverage, reduces repeated failures, saves execution time, and increases the efficiency of regression testing.

14. Tester Role and Early Collaboration

Testers play an important role in reducing integration defects by participating in development activities from the early stages of the SDLC. During requirement and design reviews, testers can identify unclear interfaces, missing conditions, data-flow problems, and integration risks before implementation. During development, testers can discuss testability and expected behavior with developers and prepare integration scenarios in advance. For example, if an e-commerce payment module must exchange data with an external payment service, testers can identify required response formats, timeout conditions, and failure handling before integration begins. Continuous communication between developers and testers reduces misunderstandings and improves defect prevention. Therefore, early tester involvement promotes collaboration, detects issues earlier, reduces integration defects, and improves overall software quality.

15. Defect Prevention

Defect prevention focuses on identifying and eliminating the root causes of defects rather than repeatedly fixing the same failures. The team should analyze recurring defects using techniques such as root-cause analysis, code reviews, requirement reviews, process improvement, checklists, and preventive regression testing. For example, if every release contains incorrect date validation, the team should determine whether the root cause is unclear requirements, inadequate coding standards, or missing boundary tests. A permanent regression test should then be added, and the development process should be improved to prevent the issue from recurring. Defect trends from the repository can also be reviewed after each release. Therefore, defect prevention reduces recurring failures, decreases maintenance costs, improves development processes, and increases long-term software quality.

16. Structured Testing for Mobile Application

Frequent crashes caused by inconsistent testing indicate that the mobile application requires a standardized testing strategy. The team should define common procedures for requirement analysis, test case design, execution, defect reporting, regression testing, and test closure. Testing should cover different devices, operating systems, screen sizes, network conditions, battery conditions, permissions, interruptions, and application states. For example, a banking mobile application should be tested during network loss, incoming calls, screen rotation, session timeout, and repeated transactions. Standard test-case templates and

defect-reporting formats should be used across the team, while automation can support repetitive regression testing. A consistent and risk-based approach improves coverage and repeatability. Therefore, structured testing reduces missed scenarios, detects crashes earlier, and improves the reliability of the mobile application.

17. Testing vs Debugging

Testing and debugging have different purposes. Testing is the process of executing and evaluating software to identify failures, whereas debugging is the process of locating and correcting the root cause of a detected defect. For example, if a mobile banking application crashes when transferring money, the tester reproduces the crash, records the steps, and reports the failure. The developer then examines logs, code, database operations, and transaction handling to identify the cause and fix it. After the fix, the tester performs retesting to verify the correction and regression testing to ensure that other functions have not been affected. Confusing testing with debugging can delay defect resolution because the responsibilities become unclear. Therefore, testing discovers failures, debugging removes their causes, and effective coordination between both activities improves software quality.

18. Cross-Browser Testing

Cross-browser testing ensures that a web application provides consistent functionality and appearance across different browsers, versions, operating systems, and devices. The testing team should identify the browsers commonly used by customers and create a browser compatibility matrix. Important functions such as login, product search, shopping cart, checkout, payment, and order tracking should be tested across the selected environments. For example, if a checkout button works correctly in Chrome but is not displayed in Firefox, the issue should be recorded with the browser version, operating system, steps to reproduce, expected result, and actual result. Automation tools such as Selenium, Playwright, or similar tools can improve coverage, while manual testing can validate visual and usability issues. Therefore, systematic cross-browser testing detects compatibility problems before customers experience them and improves the consistency and reliability of the shopping system.

19. Validation Gaps in a Logistics Application

Inconsistent outputs in a logistics application may occur because input validation is incomplete or incorrect. The testing team should identify missing validation scenarios such as empty fields, invalid tracking numbers, incorrect dates, negative quantities, duplicate entries, invalid addresses, and unexpected characters. Equivalence partitioning can divide valid and invalid inputs into groups, while boundary value analysis can test minimum and maximum limits. For example, if a package weight must be between 0.1 kg and 100 kg,

values below 0.1, exactly 0.1, normal values, exactly 100, and above 100 should be tested. Negative testing should verify that invalid inputs are rejected with meaningful error messages. Proper validation prevents incorrect data from entering the system and reduces downstream failures. Therefore, systematic input validation improves reliability, consistency, data quality, and robustness of the logistics application.

20. Systematic Test Case Design

Random testing may miss important conditions because there is no guarantee that critical requirements, boundaries, or exceptional scenarios will be covered. Testing should therefore be treated as an engineering activity based on requirements, risks, and systematic test design. Test cases should include preconditions, inputs, execution steps, expected results, actual results, and status. Techniques such as equivalence partitioning, boundary value analysis, decision tables, state transition testing, and use-case testing can be used. For example, for a login system, testers should cover valid credentials, invalid username, invalid password, empty fields, locked accounts, expired passwords, and repeated failed attempts. Requirements traceability should ensure that important requirements have corresponding test cases. Therefore, systematic test design provides measurable coverage, improves defect detection, reduces random duplication, and produces more reliable testing results.

21. Data Consistency Between Modules

Data inconsistency between healthcare modules is commonly an integration and data-integrity defect. The root cause may be incorrect data mapping, database synchronization problems, inconsistent formats, incorrect API responses, transaction failures, or improper handling of updates. For example, if a patient's address is updated in the registration module but the old address continues to appear in the billing module, the integration is not functioning correctly. Testers should perform integration testing to verify data transfer between modules and database validation to ensure that the correct records are stored and retrieved. Positive, negative, concurrency, and transaction-rollback scenarios should also be tested. Automated regression tests can verify consistency after future changes. Therefore, systematic integration and data validation testing ensures that information remains accurate and synchronized across all modules.

22. Tester Role Across SDLC

Testers should contribute throughout the SDLC rather than entering the project only after coding is complete. During requirements, they review ambiguity and testability; during design, they identify risks and prepare test scenarios; during development, they support unit and integration testing; during system testing, they validate complete functionality; and during maintenance, they perform regression testing after changes.

For example, in a banking system, early tester involvement can identify missing requirements related to transaction limits, failed transactions, and security before coding begins. This prevents defects from being introduced and reduces the cost of correcting them later. Continuous collaboration between testers, developers, analysts, and stakeholders improves communication and creates shared understanding of quality expectations. Therefore, tester involvement across SDLC promotes early defect detection, reduces rework, improves test coverage, and produces higher-quality software.

23. Testing for Peak Usage

A system that fails during peak usage indicates insufficient performance and scenario coverage. Testing should identify realistic peak conditions and include load, stress, scalability, and concurrency testing. For example, if an online examination system normally supports 1,000 users but may receive 10,000 simultaneous logins, the testing team should simulate this workload before deployment. Testers should measure response time, throughput, CPU and memory usage, database performance, and error rates. Critical scenarios such as simultaneous login, submission, payment, file upload, and database transactions should be included. Boundary and negative scenarios should also be considered because failures may occur when system resources become limited. Therefore, realistic performance testing and comprehensive test coverage can identify bottlenecks before deployment and prevent failures during peak usage.

24. Defect Documentation and Repository

A defect repository provides a centralized system for recording, tracking, analyzing, and resolving software defects. The company should use a standard defect format containing defect ID, title, module, environment, steps to reproduce, expected result, actual result, severity, priority, screenshots or logs, assigned developer, root cause, fix, retest result, and closure status. For example, instead of writing “login problem,” the tester should clearly document the exact steps that cause the login failure and the difference between expected and actual behavior. A properly maintained repository also allows teams to identify recurring defects, high-risk modules, and common root causes. This information can be reused to create regression tests and improve future development practices. Therefore, proper defect documentation improves traceability, communication, accountability, analysis, and overall defect-management effectiveness.

25. Structured Testing for Banking Transactions

Repeated transaction failures indicate that the banking system requires a structured and risk-based testing process. Requirements should first be analyzed for transaction rules, authentication, account balance, limits, database updates, and failure handling. Test cases should then cover valid transactions as well as insufficient balance, invalid accounts, maximum and minimum amounts, duplicate transactions, network failures,

timeouts, and transaction cancellation. Integration testing should verify communication between the application, database, authentication system, and banking server. Regression testing should be automated for previously fixed defects, while performance and security testing should validate the system under realistic conditions. Defect repository data should be analyzed to identify modules with recurring failures. Therefore, structured testing improves coverage, detects transaction defects earlier, reduces defect leakage, and increases the reliability and security of banking services.

26. Edge Case and Boundary Testing

Edge-case failures occur when testers focus mainly on normal inputs and fail to test unusual or extreme conditions. Testing should deliberately include minimum values, maximum values, values just below and above limits, empty inputs, maximum-length inputs, zero values, invalid formats, and unexpected sequences of actions. Boundary Value Analysis is particularly useful because software defects frequently occur near input boundaries. For example, if an application accepts a password length from 8 to 20 characters, testers should check 7, 8, 9, 19, 20, and 21 characters. These cases can reveal validation and processing errors that normal inputs may not expose. Combining boundary testing with equivalence partitioning and negative testing improves coverage of unusual conditions. Therefore, systematic edge-case testing increases defect detection and reduces failures after deployment.

27. Process-Based Testing

Process-based testing establishes standardized procedures so that testing is performed consistently across all modules and releases. The organization should define common practices for requirement analysis, test planning, test-case preparation, execution, defect reporting, retesting, regression testing, and test closure. For example, every module in an e-commerce application should use a standard test-case format and defect-severity classification while still having module-specific scenarios. Checklists, templates, review procedures, entry and exit criteria, and requirement traceability can ensure consistency. Regular test reviews and process audits can identify deviations from the defined standards. Standardization reduces missed activities, improves communication, increases repeatability, and makes test results easier to compare across modules. Therefore, process-based testing creates consistent quality practices and improves the overall reliability of the software.

28. Input Validation and Robustness

A web application that fails when unexpected inputs are provided has inadequate input validation. Testers should identify both valid and invalid input classes and verify that the application handles unexpected values safely without crashing or producing incorrect results. Testing should include empty values, special

characters, extremely long strings, incorrect data types, negative numbers, duplicate values, invalid formats, and boundary values. For example, if an age field accepts values from 18 to 60, inputs such as letters, negative numbers, 17, 18, 60, and 61 should be tested. The system should reject invalid data with clear messages instead of crashing. Equivalence partitioning, boundary value analysis, negative testing, and security-oriented input validation can improve robustness. Therefore, systematic validation prevents invalid data from reaching internal processing and makes the application more stable and reliable.

29. Defect Leakage into Production

Defect leakage occurs when defects are not detected during testing and reach the production environment. Frequent leakage may result from incomplete requirements, inadequate test coverage, insufficient negative and boundary testing, weak regression testing, poor defect analysis, or lack of realistic test environments. The team should analyze production defects to identify their root causes and update test cases accordingly. For example, if a payment defect repeatedly reaches production because timeout conditions were never tested, timeout and network-failure scenarios should be permanently added to the regression suite. Risk-based testing should prioritize critical business functions, while automation should support frequent regression execution. Requirements traceability, independent reviews, realistic test data, and proper exit criteria should also be used. Therefore, systematic testing, historical defect analysis, and continuous improvement can significantly reduce defect leakage and improve production quality.

30. Early Defect Detection and Testing Strategy

Early defect detection improves efficiency because defects are generally easier and less expensive to correct when they are identified close to the stage in which they are introduced. The testing strategy should therefore begin with requirement reviews and continue through design, coding, integration, system, and acceptance testing. Testers should identify ambiguous requirements, prepare test scenarios early, and use techniques such as equivalence partitioning, boundary value analysis, decision tables, and risk-based testing. For example, detecting an incorrect fund-transfer requirement during requirement review is much easier than discovering it after deployment when database transactions and user interfaces have already been implemented. Automated unit, integration, and regression tests can provide continuous feedback, while defect-trend analysis can identify high-risk areas. Therefore, early and systematic testing reduces rework and maintenance costs, improves defect detection efficiency, and results in higher-quality software.



Code of Conduct:

I R.Rachitha/192421058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Rachitha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand